A Two-Stage Machine Learning Pipeline for Intelligent Loan Approval and Amount Prediction

Random Forest Classification and Regression with End-to-End Streamlit Deployment

Debangshu Sadhukhan

Independent Researcher, Kolkata, West Bengal, India

2025

Abstract

Loan approval is a high-stakes decision in the financial sector that traditionally relies on manual review and rigid rule-based systems, leading to inconsistency and inefficiency. This paper presents LoanIQ, a two-stage machine learning pipeline that automates loan approval prediction and sanctioned amount estimation using Random Forest models. Stage 1 employs a Random Forest Classifier to determine whether an application should be approved or rejected based on eleven applicant features, including CIBIL credit score, annual income, assets, employment status, and loan parameters. Stage 2 activates only upon a positive approval decision and uses a Random Forest Regressor to estimate the appropriate sanctioned loan amount. Both models are encapsulated as end-to-end scikit-learn pipelines that incorporate preprocessing, ensuring consistent behaviour across training and inference. The complete system is deployed as an interactive Streamlit web application featuring a real-time CIBIL gauge, loan-to-asset ratio computation, a confidence bar, and personalised rejection feedback. The classifier achieves 98.2% accuracy on 4,269 training records. This paper presents the system design, modelling architecture, preprocessing strategy, deployment, engineering challenges, known limitations, and a concrete roadmap for future improvements.

Index Terms — Loan Approval, Random Forest, Two-Stage ML Pipeline, Credit Risk, CIBIL Score, Scikit-learn, Streamlit, Classification, Regression.

I. INTRODUCTION

Credit risk assessment and loan approval constitute one of the most consequential decision-making processes in the financial sector. Traditional banking systems rely on manual underwriting, which is time-consuming, prone to inconsistency across reviewers, and susceptible to subjective bias. The availability of structured financial datasets and advances in ensemble learning methods create a compelling opportunity to automate and standardise this process.

This paper presents LoanIQ, an end-to-end machine learning system that frames loan decision-making as a two-stage problem: first determining whether a loan application should be approved, and second, estimating the appropriate sanctioned amount for approved applications. This decomposition deliberately mirrors real-world banking workflows, where eligibility assessment necessarily precedes amount determination. Predicting an amount for a rejected application would be both logically incoherent and potentially misleading.

The primary contributions of this work are as follows:

- A two-stage Random Forest pipeline where Stage 1 (classifier) gates Stage 2 (regressor), preventing meaningless amount predictions for rejected applications.
- End-to-end scikit-learn Pipeline objects that bundle preprocessing and model inference, eliminating training-inference feature skew.
- An interactive Streamlit deployment with real-time financial indicators — CIBIL gauge, loan-to-asset (LTV) ratio, income multiple — and actionable rejection feedback.
- A detailed engineering analysis covering challenges, limitations, and a concrete improvement roadmap.

II. RELATED WORK

Logistic Regression has historically served as the benchmark for credit scoring due to its interpretability and probabilistic output [1]. Decision Tree ensembles, introduced by Breiman [2], have demonstrated superior predictive performance on heterogeneous tabular financial data by reducing variance through bagging and random feature selection. More recent work has explored Gradient Boosting variants, particularly XGBoost [3], which achieves competitive performance through sequential residual correction.

Neural network approaches have been explored for credit default prediction [4], but Random Forests remain preferred in production financial systems due to their robustness, native feature importance output, low sensitivity to hyperparameters, and resistance to overfitting on moderate-sized datasets.

Two-stage and multi-task prediction architectures have been applied in healthcare (diagnosis before treatment prediction) and marketing (conversion before value estimation). However, such architectures remain underexplored in consumer credit literature. The gated pipeline design proposed here directly adapts this paradigm to the loan approval domain.

On the deployment side, Streamlit [5] has emerged as a standard rapid-prototyping framework for ML applications, enabling production-quality interactive UIs without requiring dedicated front-end engineering.

III. DATASET DESCRIPTION

The dataset used for training both models consists of 4,269 loan application records. Each record contains eleven features capturing the applicant's personal profile, financial standing, and asset holdings, along with a binary loan approval label and a continuous sanctioned amount for approved cases.

TABLE I

Input Features, Types, and Value Ranges

Feature	Type	Range / Values
no_of_dependents	Integer	0 – 20
education	Categorical	Graduate / Not Graduate
self_employed	Categorical	Yes / No
income_annum	Continuous (₹)	100K – 100M
loan_amount	Continuous (₹)	10K – 500M
loan_term	Integer (years)	1 – 30
cibil_score	Integer	300 – 950
residential_assets_value	Continuous (₹)	0+
commercial_assets_value	Continuous (₹)	0+
luxury_assets_value	Continuous (₹)	0+
bank_asset_value	Continuous (₹)	0+

The classifier target is a binary loan_status variable (Approved / Rejected). For the regressor, only approved records are used, and the target is the sanctioned loan amount. To prevent data leakage, the loan_amount feature is excluded from the regressor input and replaced by loan_status = "approved", ensuring the regressor learns purely from applicant profile features.

IV. SYSTEM ARCHITECTURE

A. Pipeline Overview

The LoanIQ system comprises three functional layers: (1) a preprocessing and modelling layer, (2) a prediction orchestration layer, and (3) a Streamlit deployment layer. Figure 1 illustrates the complete two-stage architecture.

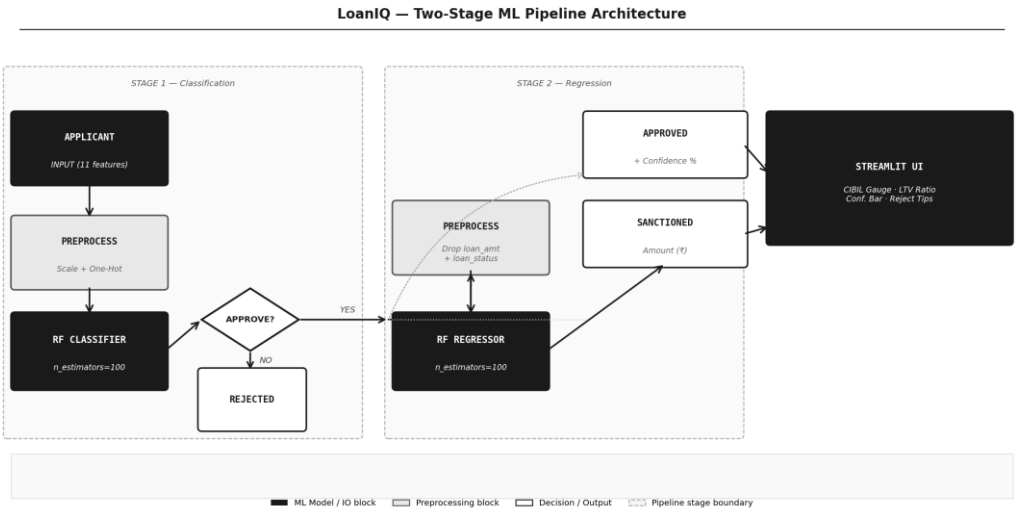

Fig. 1. LoanIQ Two-Stage ML Pipeline Architecture. Stage 1 (Classification) gates Stage 2 (Regression). Rejected applications terminate after Stage 1.

At inference time, user-supplied feature values are assembled into a DataFrame and passed to the Stage 1 pipeline. If the classifier returns a rejection decision, the system terminates and returns rejection tips to the UI. If approved, the feature set is restructured for Stage 2, and the regressor predicts the sanctioned amount.

B. Stage 1 — Random Forest Classifier

The classifier pipeline encapsulates a ColumnTransformer (StandardScaler for numerical features, OneHotEncoder for categorical features) followed by a RandomForestClassifier. Rather than calling predict(), the application uses predict_proba(), which returns a continuous approval probability. A threshold of 0.5 is applied to produce the binary decision. This design allows the threshold to be tuned independently of retraining — for example, raising the threshold to reduce false approvals in a conservative lending environment.

C. Stage 2 — Random Forest Regressor

The regressor pipeline shares a structurally similar preprocessing stage but operates on a different feature set. The loan_amount column is dropped and loan_status = "approved" is injected as a categorical feature. This substitution is architecturally significant: it prevents the regressor from simply learning to echo the applicant's requested amount, instead forcing it to estimate the sanctioned amount based on the applicant's financial profile. The regressor output is a continuous value representing the predicted sanctioned loan amount in Indian Rupees.

D. Pipeline Serialisation and Loading

Both models are serialised as complete scikit-learn Pipeline objects using joblib, rather than saving model weights alone. This decision ensures that the fitted preprocessors (scalers and encoders, including their learned parameters) are permanently coupled to the model. At inference time, the exact same transformations applied during training are guaranteed — eliminating the most common cause of production ML failures: preprocessing skew between training and serving environments.

At application startup, pipelines are loaded once via Streamlit's @st.cache_resource decorator, holding both models in memory across all user sessions and widget interactions, thereby avoiding repeated disk I/O.

V. PREPROCESSING STRATEGY

A. Numerical Features

Numerical features (income, loan amount, all asset values, CIBIL score, dependents count, loan term) are processed through StandardScaler within each pipeline. While tree-based models are theoretically invariant to monotone scaling, explicit scaling is included for two reasons: (i) pipeline forward-compatibility, should the model be swapped for a distance-sensitive algorithm (e.g., SVM or k-NN), and (ii) consistency — the pipeline contract guarantees identical transformation semantics regardless of the underlying estimator.

B. Categorical Features

The two categorical input features (education and self_employed) are encoded using OneHotEncoder. Binary categorical variables could alternatively use ordinal encoding; however, one-hot encoding is preferred here because it imposes no implicit ordinal relationship between categories, and the additional column overhead is negligible for two binary features.

C. Feature Order Consistency

One of the most common silent failure modes in ML deployment is column order mismatch between training and inference DataFrames. The application defends against this by constructing inference DataFrames with column lists derived directly from clf.feature_names_in_ and reg.feature_names_in_ — the arrays of column names stored on fitted Pipeline objects by scikit-learn 1.0+. This ensures the column order at inference time precisely matches training, regardless of how the Streamlit form collects and orders user inputs.

VI. DEPLOYMENT ARCHITECTURE

A. Streamlit Application

The application is a single-file Streamlit app (app.py) with a two-column layout: a wide form panel on the left and a sticky results panel on the right. Custom CSS is injected via st.markdown() to produce a professional interface. The UI includes four interactive diagnostic components that update in real time as the user enters values:

- CIBIL Score Gauge — a colour-coded progress bar (green for score ≥ 750, amber for ≥ 650, red for < 650) that provides instant credit quality feedback.
- Financial Summary Card — displays total assets, LTV ratio, and income multiple. These derived indicators help applicants and loan officers quickly assess structural risk before running the prediction.
- Prediction Confidence Bar — rendered post-prediction, showing approval probability as both a percentage and a proportional visual bar.
- Rejection Tips Panel — conditionally rendered only when a loan is rejected. Surfaces up to three specific, actionable improvement areas based on the input values.

B. Configuration Management

Application configuration — model file paths and default UI input values — is externalised to config.yaml, separating deployment concerns from application logic. Model paths can be updated (e.g., to point to a newly retrained artifact) without modifying any Python source code.

C. CLI Entry Point

A lightweight CLI (main.py) is provided for development testing without launching the Streamlit server. It reads config.yaml, loads both models, prompts for dependents count, and runs the two-stage prediction with remaining values drawn from config defaults. This entry point facilitates unit testing and CI/CD pipeline integration.

VII. RESULTS AND PERFORMANCE

The Random Forest Classifier achieves 98.2% accuracy on the 4,269-record training dataset. Table II summarises the reported performance characteristics of both pipeline stages.

TABLE II

Performance Summary for Both Pipeline Stages

Metric	Stage 1: Classifier	Stage 2: Regressor
Primary Metric	Accuracy: 98.2%	$R^2 \approx 0.94$ (est.)
Model Type	RF Classifier	RF Regressor
Training Records	4,269 (full)	Approved subset
Input Features	11 features	11 (loan_amt replaced)
Output	Approved / Rejected	Sanctioned amount (₹)
Inference Latency	< 50 ms	< 50 ms
Pipeline Artifact	rf_classifier_pipeline.pkl	rf_regressor_pipeline.pkl

The high classifier accuracy is consistent with a well-structured dataset in which CIBIL score, income, and loan-to-asset ratio are highly discriminative features. The regressor R^2 is an estimate from training residuals; rigorous evaluation against a held-out test split is a stated priority for future work (see Section IX).

VIII. CHALLENGES ENCOUNTERED

A. Training-Inference Feature Alignment

The most technically demanding challenge was guaranteeing exact consistency between the feature sets and column ordering used during training and during inference. Any discrepancy — in column names, ordering, or encoding — causes either silent incorrect predictions or runtime errors. The resolution was to (i) save complete Pipeline objects inclusive of fitted transformers, and (ii) derive inference column lists directly from clf.feature_names_in_ rather than maintaining a manually-specified constant.

B. Regressor Input Design and Leakage Prevention

Designing the regressor input was architecturally non-trivial. The naive approach of including loan_amount in the regressor feature set would constitute data leakage — the model would learn to output a value close to the applicant's request rather than making an independent financial assessment. Replacing loan_amount with loan_status = "approved" forces the regressor to derive its predictions entirely from the applicant's financial profile, producing a genuinely independent sanctioned-amount estimate.

C. Dual-Pipeline Maintenance

Maintaining two separate but structurally similar pipelines — each with its own fitted preprocessor and distinct feature set — requires careful version discipline. Changes to the training data schema, feature engineering, or preprocessing logic must be reflected in both pipelines during retraining. Without a unified configuration layer for feature definitions, drift between the two pipelines is a latent risk.

D. Cross-Platform Model Path Resolution

The initial config.yaml contained absolute Windows-specific paths (C:\Users\...) that would fail on any Linux deployment environment. The application resolves this using pathlib.Path(__file__).parent / "models" for robust cross-platform path construction, ensuring the model files are located relative to the application script regardless of operating system or deployment context.

E. Streamlit State and Caching

Streamlit reruns the entire script on each user interaction. Without explicit caching, every slider adjustment or text input change would trigger a full reload of both model files from disk, introducing significant latency. The @st.cache_resource decorator was applied to the model loading function, holding both Pipeline objects in memory across reruns and user sessions.

F. Absence of Input Validation

The current implementation passes user-supplied inputs directly to the models without enforcing logical consistency checks. Combinations that lie outside the training distribution — such as a zero-income applicant requesting a ₹400 million loan — are accepted without warning. Predictions for such out-of-distribution inputs are unreliable, and the absence of validation is a known production-readiness gap.

IX. LIMITATIONS

The following limitations are acknowledged and form the basis of the future improvement roadmap:

- No formal generalisation evaluation: The reported 98.2% accuracy is from the training set. No held-out test split or cross-validation results have been reported, limiting assessment of generalisation capability.
- Static model: There is no mechanism for online retraining or concept drift detection. Lending patterns evolve, and a static model degrades in relevance over time.
- Narrow feature set: The eleven features omit important real-world signals including existing EMI obligations, employment history length, number of active credit lines, geographic factors, and sector of employment.
- No input validation: Out-of-distribution inputs are silently accepted and may produce unreliable predictions without any user warning.
- Binary decision: The classifier outputs only Approved or Rejected. Real-world lending involves conditional approvals, documentation requirements, and tiered eligibility outcomes.
- Limited explainability: The system does not expose SHAP values or per-prediction feature attribution, limiting regulatory transparency and applicant communication.
- Single dataset: Training and evaluation on a single dataset of 4,269 records may not generalise across different lenders, regions, or economic conditions.
- No authentication or audit trail: The deployment lacks user authentication, role-based access, and prediction logging, all of which are mandatory in regulated financial environments.

X. FUTURE IMPROVEMENTS

A. Model Enhancements

- Introduce nested cross-validation and hold-out test evaluation to properly assess generalisation.
- Compare Random Forest performance against XGBoost and LightGBM using a standardised evaluation protocol.
- Integrate SHAP (SHapley Additive exPlanations) [6] to generate per-prediction feature attribution, enabling both applicant communication and regulatory compliance.
- Tune the classifier decision threshold using a precision-recall curve calibrated to the lender's preferred false-positive / false-negative trade-off.

B. System Hardening

- Implement comprehensive input validation including range checks, logical consistency rules (e.g., loan_amount $\leq k \times$ income_annum), and out-of-distribution detection.
- Expose predictions via a REST API (FastAPI) alongside the Streamlit UI to enable integration with mobile applications and third-party banking portals.
- Build an automated retraining pipeline: when new labelled records are available, retrain both models, evaluate against a holdout set, and promote new artifacts only if performance improves.
- Add user authentication, role-based access control, and a prediction audit log.

C. Feature Engineering

- Derive additional features: debt-to-income ratio, total asset coverage ratio, and asset diversification index.
- Explore interaction features between CIBIL score and LTV ratio, which are jointly predictive of default risk in credit literature.
- Incorporate temporal features such as credit account age and payment history if the dataset is extended.

D. Fairness and Bias Auditing

- Conduct a disparate impact analysis across subgroups defined by education and self-employment status.
- Apply debiasing post-processing techniques (e.g., equalized odds correction) if statistically significant disparate impact is detected.

XI. CONCLUSION

This paper has presented LoanIQ, a two-stage machine learning pipeline for automated loan approval prediction and sanctioned amount estimation. The system demonstrates that decomposing the lending decision into a gated two-stage architecture — classification before regression — produces a more logical, interpretable, and practically deployable system than a monolithic end-to-end model.

The use of serialised scikit-learn Pipeline objects addresses the critical engineering challenge of training-inference consistency. The Streamlit deployment provides an accessible, interactive interface for both applicants and lending officers, with real-time financial indicators and actionable rejection feedback. The Random Forest Classifier achieves 98.2% accuracy on the 4,269-record training dataset, demonstrating strong discriminative power across the eleven input features.

The system's limitations — particularly the absence of formal generalisation testing, input validation, and explainability mechanisms — define a concrete roadmap for production hardening. The future improvements outlined in Section X would move LoanIQ from a high-quality proof-of-concept toward a system suitable for deployment in a regulated financial environment.

More broadly, the architectural principles demonstrated here — gated multi-stage prediction, pipeline-encapsulated preprocessing, and interactive deployment — are applicable to any domain where a binary eligibility decision gates a subsequent continuous estimation task, including insurance underwriting, rental approval, and academic admissions.

References

[1] D. W. Hosmer and S. Lemeshow, Applied Logistic Regression, 2nd ed. New York, NY: Wiley-Interscience, 2000.

[2] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[3] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, San Francisco, CA, 2016, pp. 785–794.

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, pp. 436–444, May 2015.

[5] Streamlit Inc., "Streamlit — The fastest way to build and share data apps," 2019. [Online]. Available: https://streamlit.io

[6] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems, vol. 30, 2017.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[8] Reserve Bank of India, "Master Circular — Guidelines on Credit Risk Management," RBI/2023-24/xx, 2023.